



UNIVERSIDAD DE CONCEPCIÓN

FACULTAD DE INGENIERÍA

DEPARTAMENTO DE INGENIERÍA INFORMÁTICA

Informe final

Integrantes:

Cristobal Figueroa

Ignacio Soto

Leonardo Guerrero

Martin Henriquez

Asignatura: Diseño de software

Profesor: Gonzalo Eduardo Rojas Duran

Profesor auxiliar: Gonzalo Eduardo Perez Correa

Fecha: 8 de julio de 2026

Índice

1. Introducción	2
2. Diseño inicial	3
2.1. Requisitos funcionales y no funcionales	3
2.2. Modelos de dominio y entidad relación	4
2.3. Diagramas de comunicación	5
2.4. Modelo C4	6
2.5. Esquema de seguridad y diseño BPMN	7
3. Resultados y aplicación	9
3.1. Flujo del sistema y módulos	9
3.1.1. Módulo de autenticación	10
3.1.2. Módulo de usuarios	10
3.1.3. Módulo de campañas	10
3.1.4. Módulo de citas	10
3.1.5. Organización interna del backend	10
3.2. Patrones de diseño: ajustes respecto al diseño inicial	11
3.3. Seguridad y autenticación	12
3.3.1. Autenticación mediante JWT	12
3.3.2. Protección del servidor	12
3.3.3. Manejo centralizado de errores	13
3.4. Interfaz de usuario y Heurísticas	13
3.4.1. Aplicación de heurísticas de usabilidad	13
3.5. Arquitectura Cliente - Servidor	14
3.6. Persistencia de datos	14
3.7. Integración de notificaciones mediante Brevo	14
4. Conclusión	16

1. Introducción

En el ámbito de la salud pública, la ejecución de campañas de vacunación constituye una tarea de gran relevancia debido a la cantidad de personas involucradas, la diversidad de instituciones participantes y la necesidad de coordinar eficientemente recursos humanos, infraestructura e información. Estas campañas buscan responder tanto a programas de inmunización periódicos como a contingencias sanitarias, proteger a grupos de riesgo y mantener un seguimiento confiable del estado de vacunación de la población. En este contexto, contar con un sistema de información centralizado que permita administrar campañas, coordinar los distintos centros de vacunación y registrar oportunamente la información clínica resulta fundamental para apoyar la toma de decisiones y optimizar el uso de los recursos disponibles.

Con el propósito de abordar esta problemática, durante el desarrollo del curso se diseñó e implementó un sistema de gestión de campañas de vacunación aplicando los principios, metodologías y buenas prácticas de la Ingeniería de Software. El proyecto evolucionó desde una etapa inicial de análisis y diseño, en la que se definieron los requerimientos, el modelo de dominio, la arquitectura mediante el modelo C4, diagramas UML, diagramas de comunicación y procesos BPMN, hacia una implementación funcional refinada a partir de la retroalimentación obtenida en la entrega anterior.

La solución desarrollada permite administrar campañas de vacunación, gestionar usuarios con distintos roles, registrar personas y sus antecedentes de vacunación, coordinar la disponibilidad de centros de vacunación y horarios, agendar citas para ciudadanos y mantener un historial de las vacunas administradas. Asimismo, incorpora una base de datos relacional para el almacenamiento persistente de la información y utiliza una API externa de correo electrónico para enviar notificaciones relacionadas con el proceso de agendamiento, integrando así servicios externos que enriquecen la funcionalidad del sistema.

La aplicación desarrollada integra los principales módulos necesarios para gestionar el ciclo completo de una campaña de vacunación. Entre sus funcionalidades se encuentran la administración de campañas, centros de vacunación, horarios y usuarios; el registro de personas; el agendamiento de citas; el registro de las vacunaciones realizadas; y la consulta del historial de vacunación. Además, el sistema implementa mecanismos de autenticación y seguridad para el acceso de los distintos tipos de usuarios, incorpora una base de datos relacional para el almacenamiento persistente de la información e integra una API externa de mensajería para el envío automático de notificaciones asociadas al proceso de agendamiento.

Durante la implementación se aplicaron distintos principios y patrones de diseño orientados a objetos con el objetivo de obtener una solución modular, mantenible y extensible. Asimismo, la interfaz de usuario fue diseñada y refinada considerando heurísticas de usabilidad, permitiendo ofrecer una experiencia de uso más intuitiva y consistente para los distintos actores del sistema.

El presente informe describe el desarrollo de la solución implementada, las decisiones de diseño adoptadas, la arquitectura del sistema, la estructura de la base de datos, la integración con servicios externos y las mejoras incorporadas como resultado de la retroalimentación recibida. De esta manera, se presenta una aplicación funcional que satisface los requerimientos planteados y demuestra la aplicación práctica de los conceptos estudiados durante el curso.

2. Diseño inicial

2.1. Requisitos funcionales y no funcionales

La primera etapa del proyecto estuvo enfocada en la identificación y especificación de los requisitos del sistema, los cuales constituyeron la base para el diseño de la arquitectura y, posteriormente, para la implementación de la aplicación. Se definieron las necesidades del problema desde la perspectiva de los distintos actores involucrados en una campaña de vacunación, estableciendo las funcionalidades esperadas, las restricciones del sistema y los criterios de calidad que debían cumplirse. Esta especificación permitió orientar todas las decisiones de diseño posteriores y sirvió como referencia para validar que la solución implementada respondiera efectivamente a los objetivos planteados.

Los requisitos funcionales fueron organizados utilizando la metodología MoSCoW (Must, Should, Could y Won't), permitiendo priorizar el desarrollo de acuerdo con la importancia de cada funcionalidad dentro del sistema. Los requisitos clasificados como Must representaron el núcleo del proceso de vacunación, contemplando la gestión de campañas, el registro y consulta de personas, la administración de centros de vacunación, la asignación, modificación y cancelación de citas, el registro de vacunaciones y las validaciones necesarias para garantizar la consistencia del proceso, como la verificación de elegibilidad y la prevención de citas duplicadas. Estas funcionalidades definieron el flujo principal del sistema y fueron implementadas como parte fundamental de la solución desarrollada.

Por su parte, los requisitos clasificados como Should incorporaron funcionalidades que complementaban la operación del sistema y mejoraban la experiencia tanto de los funcionarios como de los ciudadanos. Entre ellas se encontraban la autenticación de usuarios, la consulta del historial de vacunación, el seguimiento del estado de una persona dentro de una campaña, la generación de reportes, el monitoreo del avance de las campañas y la emisión de confirmaciones de las citas agendadas. Si bien estos requisitos no eran indispensables para ejecutar el flujo principal, aportaban un mayor nivel de control, seguridad y trazabilidad, por lo que varios de ellos fueron considerados durante la implementación final.

Los requisitos clasificados como Could representaron funcionalidades de valor agregado cuya implementación dependía de la disponibilidad de tiempo y recursos. En este grupo se incluyeron mecanismos de notificación automática, integración con fuentes de datos externas y administración avanzada de roles y permisos. Como parte de la evolución del proyecto, una de estas funcionalidades fue incorporada mediante la integración de una API externa de mensajería para el envío de correos electrónicos de confirmación, permitiendo automatizar parte de la comunicación con los usuarios y acercar la aplicación a un escenario de uso más realista. Finalmente, los requisitos definidos como Won't establecieron explícitamente el alcance del proyecto, dejando fuera características como analítica avanzada, autogestión de cuentas por parte de los ciudadanos o segmentaciones complejas de campañas, evitando así incrementar innecesariamente la complejidad de la solución.

En complemento a las funcionalidades del sistema, se definieron requisitos no funcionales cuyo propósito fue establecer los atributos de calidad que debía cumplir la aplicación. Estos requisitos sirvieron como guía para la construcción de una solución robusta, segura y mantenible, más allá de las operaciones que el sistema debía realizar. En particular, se establecieron criterios relacionados con la seguridad, el rendimiento, la fiabilidad, la mantenibilidad, la usabilidad y la disponibilidad, aspectos que influyeron tanto en las decisiones de diseño como en la implementación de los distintos componentes del sistema.

Desde el punto de vista de la seguridad, se planteó la necesidad de proteger la información personal mediante mecanismos de autenticación, autorización basada en roles y resguardo de los datos almacenados. Estos principios se reflejaron en la implementación de un sistema de autenticación que restringe el acceso a las funcionalidades según el perfil del usuario y protege las operaciones críticas realizadas dentro de la aplicación. Asimismo, se consideró la importancia de mantener la integridad de la información mediante validaciones que impiden inconsistencias en la asignación de citas y en el registro de vacunaciones.

En cuanto al rendimiento y la escalabilidad, los requisitos buscaron asegurar que el sistema pudiera responder adecuadamente frente a escenarios de alta demanda, característicos de campañas de vacunación masivas. Aunque el proyecto se desarrolló con fines académicos y no fue sometido a pruebas de carga reales, la

arquitectura modular y la separación de responsabilidades permitieron construir una solución preparada para evolucionar y soportar un crecimiento futuro sin afectar significativamente su funcionamiento.

Respecto de la mantenibilidad y extensibilidad, los requisitos promovieron una arquitectura organizada y desacoplada, favoreciendo la incorporación de nuevas funcionalidades sin modificar de manera significativa los componentes existentes. Este objetivo se vio reflejado en la utilización de patrones de diseño y en la separación entre las capas de presentación, lógica de negocio y persistencia, facilitando la evolución del sistema durante las iteraciones realizadas tras la retroalimentación recibida. De igual forma, la integración con una API externa para el envío de correos demostró la capacidad del sistema para incorporar servicios externos sin alterar el funcionamiento del resto de la aplicación.

Finalmente, los requisitos relacionados con la usabilidad y la disponibilidad orientaron el diseño de la interfaz y de la experiencia de usuario. La aplicación fue refinada considerando heurísticas de usabilidad con el propósito de simplificar la interacción de ciudadanos y funcionarios durante el proceso de agendamiento y registro de vacunaciones. Paralelamente, la utilización de una base de datos relacional permitió garantizar la persistencia de la información y mantener la consistencia de los datos registrados a lo largo de todo el flujo del sistema.

En conjunto, los requisitos funcionales y no funcionales no solo establecieron el alcance inicial del proyecto, sino que también sirvieron como criterio para evaluar las decisiones tomadas durante el diseño y la implementación.

2.2. Modelos de dominio y entidad relación

Tras la definición de los requerimientos del sistema, la siguiente etapa del desarrollo consistió en modelar el problema desde una perspectiva conceptual y de persistencia mediante el modelo de dominio y el modelo entidad-relación. Ambos modelos fueron desarrollados de manera complementaria, representando las principales entidades del sistema, sus relaciones y las reglas de negocio que sustentan el proceso de gestión de campañas de vacunación. Mientras el modelo de dominio permitió estructurar la lógica de negocio desde un enfoque orientado a objetos, el modelo entidad-relación tradujo dicha representación hacia una estructura adecuada para el almacenamiento de la información en una base de datos relacional. En conjunto, estos modelos sirvieron como puente entre el análisis de requisitos y la implementación final de la aplicación.

El diseño se organizó en torno a los principales procesos del sistema: la administración y seguridad, la planificación de campañas y el proceso de atención clínica. Como núcleo del modelo se definió la entidad Persona, responsable de centralizar la información de los individuos que participan en el sistema. A partir de ella se diferenciaron conceptualmente ciudadanos y funcionarios, evitando duplicidad de información y permitiendo representar comportamientos específicos de cada tipo de actor. Además, se estableció una separación entre la identidad de una persona y su acceso al sistema mediante la incorporación de la entidad Usuario, asociada directamente a un Rol, decisión que simplificó la implementación del mecanismo de autenticación y autorización basado en perfiles de usuario desarrollado en la aplicación.

La planificación de las campañas de vacunación se modeló mediante las entidades Campaña, Punto de Vacunación, Horario y Criterio de Elegibilidad, las cuales representan la organización previa al proceso de atención. Cada campaña agrupa las actividades de inmunización dentro de un período determinado y establece las condiciones que deben cumplir las personas para acceder a ella. Con el fin de mantener la flexibilidad del sistema, los criterios de elegibilidad fueron modelados como una entidad independiente, evitando incorporarlos como atributos fijos de una campaña y permitiendo extender fácilmente las condiciones de participación sin modificar la estructura general del modelo.

Para representar la disponibilidad de atención se incorporó la entidad Horario, inexistente en el esquema base, permitiendo asociar cupos disponibles a cada punto de vacunación y controlar de manera consistente el proceso de asignación de citas. Esta decisión facilitó posteriormente la implementación de las validaciones necesarias para evitar sobreasignaciones y conflictos de disponibilidad durante el agendamiento.

El flujo principal del sistema se estructuró alrededor de las entidades Cita, Vacunación y Vacuna, diferenciando explícitamente el proceso administrativo de reserva de una atención del acto clínico correspondiente a la inmunización. Una cita representa la planificación de una atención para una persona en un horario y

punto de vacunación determinados, mientras que la vacunación registra la aplicación efectiva de una vacuna. Esta separación permite modelar situaciones habituales del dominio, como cancelaciones, reprogramaciones o inasistencias, manteniendo al mismo tiempo la trazabilidad de las vacunaciones efectivamente realizadas. Complementariamente, la incorporación de la entidad Vacuna permitió almacenar información propia del biológico administrado, como el laboratorio fabricante y la cantidad de dosis requeridas, independiente de la campaña en la que fue utilizada.

La construcción del modelo entidad-relación permitió transformar estas relaciones conceptuales en una estructura lógica de base de datos, definiendo claves primarias, claves foráneas, cardinalidades y restricciones de integridad necesarias para garantizar la consistencia de la información. Las entidades principales del dominio fueron preservadas dentro del esquema relacional, manteniendo una correspondencia directa entre la lógica del negocio y la persistencia de los datos, lo que facilitó considerablemente la implementación posterior mediante la base de datos utilizada por la aplicación.

Adicionalmente, tanto el modelo de dominio como el modelo entidad-relación fueron complementados mediante un conjunto de restricciones y supuestos que representan reglas del negocio difíciles de expresar únicamente mediante asociaciones y cardinalidades. Entre ellas se encuentran el control de cupos disponibles por horario, la prevención de citas duplicadas, la consistencia entre citas y vacunaciones, la administración de roles de usuario, las validaciones de elegibilidad y los distintos estados que puede presentar el proceso de vacunación. Estas reglas fueron posteriormente materializadas en la lógica de negocio y en la base de datos, permitiendo asegurar el correcto funcionamiento del sistema.

En conjunto, ambos modelos constituyeron la base conceptual sobre la cual se desarrolló la arquitectura y la implementación de la aplicación. La correspondencia existente entre el diseño conceptual, la estructura de la base de datos y las clases implementadas permitió mantener la coherencia durante todo el proceso de desarrollo, facilitando la incorporación de las mejoras derivadas de la retroalimentación recibida y dando origen a un sistema funcional, modular y alineado con los objetivos planteados para la gestión integral de campañas de vacunación.

2.3. Diagramas de comunicación

Como complemento al modelo de dominio, se elaboraron diagramas de comunicación con el propósito de representar la colaboración entre los distintos objetos del sistema durante la ejecución de casos de uso relevantes. Estos diagramas permitieron trasladar las relaciones estáticas definidas en el modelo conceptual hacia un comportamiento dinámico, mostrando cómo se distribuyen las responsabilidades entre las clases y cómo fluye la información durante la ejecución de las operaciones principales del sistema. Su construcción sirvió además para validar la asignación de responsabilidades propuesta en el diseño y orientar posteriormente la implementación de la lógica de negocio.

Los diagramas seleccionados corresponden a dos procesos fundamentales dentro del sistema de vacunación: el agendamiento de una cita y la consulta del historial de vacunación de un ciudadano. Ambos representan escenarios representativos de la aplicación, ya que abarcan tanto operaciones de modificación del estado del sistema como consultas sobre la información almacenada. En conjunto, permiten evidenciar la interacción entre las entidades del dominio y la aplicación de los principios GRASP para lograr una distribución adecuada de responsabilidades.

En ambos casos se utiliza un controlador como punto de entrada a la operación, siguiendo el patrón GRASP Controller. En lugar de que el actor interactúe directamente con las entidades del dominio, todas las solicitudes son recibidas por una clase encargada de coordinar el flujo de ejecución y delegar las tareas correspondientes a los objetos que poseen la información necesaria. Esta decisión permitió desacoplar la interfaz de usuario de la lógica del negocio y mantener una organización consistente durante la implementación de los distintos casos de uso.

En el proceso de agendamiento de una cita, el controlador recibe la solicitud junto con la información necesaria para realizar la reserva y coordina las interacciones entre las entidades involucradas. Como parte del flujo, el sistema consulta la disponibilidad del punto de vacunación mediante los objetos Horario, los cuales poseen la información referente a los cupos disponibles para cada franja horaria. Esta asignación de responsabilidades

responde al patrón GRASP Experto en Información, ya que es precisamente la entidad Horario la que dispone de los datos necesarios para determinar si una reserva puede realizarse o no. Posteriormente, una vez verificadas las condiciones de disponibilidad, la creación de la nueva cita es delegada a la entidad Campaña, aplicando el patrón GRASP Creador. Dado que una campaña administra el conjunto de citas asociadas a su proceso de vacunación, resulta natural que sea responsable de instanciar y registrar nuevas citas dentro de su contexto.

Por otra parte, el diagrama correspondiente a la consulta del historial de vacunación representa un flujo orientado principalmente a la recuperación de información. El controlador recibe la solicitud del usuario e identifica a la persona correspondiente mediante su RUT. A continuación, el objeto Ciudadano proporciona la colección de vacunaciones registradas y, para cada una de ellas, se recupera la información asociada a la Vacuna, la Campaña y el Punto de Vacunación donde fue administrada. En este escenario vuelve a evidenciarse la aplicación del patrón Experto en Información, ya que cada entidad es responsable únicamente de proporcionar los datos que administra, evitando centralizar el conocimiento en una única clase y favoreciendo un bajo acoplamiento entre los componentes del sistema.

La utilización de estos diagramas permitió verificar que las responsabilidades asignadas durante el modelado conceptual fueran coherentes con las relaciones definidas en el modelo de dominio y el modelo entidad-relación. Asimismo, facilitaron la transición hacia la implementación al establecer con claridad qué objeto debía ejecutar cada operación y cómo debían colaborar entre sí para satisfacer los distintos casos de uso.

En la implementación final, esta organización se reflejó en la estructura de servicios y controladores de la aplicación, donde cada componente conserva responsabilidades bien definidas y delega las operaciones específicas a las clases del dominio correspondientes. De esta manera, los diagramas de comunicación no solo sirvieron como documentación del comportamiento del sistema, sino también como una guía para construir una solución modular, mantenible y alineada con los principios de diseño orientado a objetos aplicados durante el desarrollo del proyecto.

2.4. Modelo C4

Con el propósito de definir la arquitectura del sistema de manera estructurada y facilitar la comprensión de la solución desde distintos niveles de abstracción, se empleó el modelo C4. Este modelo permitió representar progresivamente el sistema, comenzando por su interacción con el entorno, continuando con la organización de sus principales contenedores y finalizando con el detalle interno de uno de ellos. La utilización de este enfoque permitió mantener una visión consistente durante el desarrollo del proyecto y establecer una correspondencia clara entre el diseño arquitectónico y la implementación final de la aplicación.

En el nivel de Contexto (C1) se representó al Sistema de Gestión de Campañas de Vacunación como una plataforma central encargada de coordinar las actividades relacionadas con el proceso de inmunización. En esta vista se identificaron los principales actores externos que interactúan con la aplicación, entre ellos los ciudadanos, los funcionarios de salud y el funcionario encargado de administrar las campañas, cada uno con responsabilidades específicas dentro del proceso. Asimismo, se incorporó la interacción con los centros de salud, los cuales proporcionan la infraestructura necesaria para ejecutar las vacunaciones y mantener sincronizada la información correspondiente a la atención de los pacientes. Esta vista permitió delimitar claramente el alcance del sistema y comprender las relaciones existentes entre la plataforma y su entorno operativo.

Posteriormente, el nivel de Contenedores (C2) permitió descomponer la aplicación en los principales bloques funcionales que participan en su funcionamiento. La arquitectura propuesta separa claramente la capa de presentación, la lógica de negocio y la persistencia de la información. Desde la interfaz de usuario, implementada mediante clientes web, las solicitudes son dirigidas hacia los distintos módulos responsables de la autenticación, la gestión de campañas, el agendamiento de citas, el registro de vacunaciones, la administración de notificaciones y la integración con servicios externos. Todos estos componentes comparten una capa de persistencia encargada de administrar el acceso a la base de datos relacional, mientras que la comunicación con el servicio externo de correo electrónico permite notificar automáticamente a los usuarios sobre eventos relevantes del proceso de vacunación. Esta organización favorece un bajo acoplamiento entre componentes y facilita tanto el mantenimiento como la incorporación de nuevas funcionalidades.

Finalmente, el nivel de Componentes (C3) profundiza en la organización interna de uno de los contenedores principales del sistema, mostrando cómo se distribuyen las responsabilidades entre los distintos componentes que conforman la lógica de negocio. En este nivel se evidencia la separación entre los controladores encargados de recibir las solicitudes, los servicios responsables de implementar las reglas del negocio y los componentes de persistencia utilizados para acceder a la información almacenada. Esta estructura permitió mantener una distribución clara de responsabilidades, facilitando la aplicación de principios como la separación de responsabilidades y el desacoplamiento entre las distintas capas de la aplicación. Asimismo, la organización interna de los componentes favoreció la implementación de los patrones de diseño utilizados durante el desarrollo, permitiendo que cada módulo evolucionara de forma independiente sin afectar el funcionamiento del resto del sistema.

En conjunto, el modelo C4 constituyó la guía arquitectónica utilizada durante el desarrollo de la aplicación. La representación escalonada proporcionada por sus distintos niveles permitió mantener una visión coherente del sistema desde los actores externos hasta la organización interna de los componentes implementados, facilitando la comunicación de las decisiones de diseño y asegurando la correspondencia entre la arquitectura propuesta y la solución finalmente desarrollada. La estructura modular obtenida permitió además integrar de forma consistente la autenticación basada en roles, la persistencia mediante una base de datos relacional y la comunicación con servicios externos para el envío de notificaciones, manteniendo una arquitectura organizada, extensible y alineada con las buenas prácticas de ingeniería de software.

2.5. Esquema de seguridad y diseño BPMN

Como complemento a los modelos estructurales y de comportamiento desarrollados durante las etapas anteriores del proyecto, se elaboró un modelo BPMN para representar el flujo completo del proceso de vacunación desde la perspectiva del negocio y un esquema de seguridad para definir los mecanismos de protección de la información y control de acceso implementados en la aplicación. Ambos modelos permitieron representar aspectos que no podían expresarse completamente mediante diagramas de clases o de comunicación, proporcionando una visión integral del funcionamiento del sistema tanto desde el punto de vista operativo como de la seguridad.

El modelo BPMN describe la interacción entre los principales participantes del proceso mediante tres carriles de responsabilidad: Usuario, Sistema y Base de Datos. Esta organización permite visualizar claramente cómo se distribuyen las actividades durante el ciclo de vida de una reserva de vacunación y cómo intervienen cada uno de los componentes involucrados.

El flujo comienza con el acceso del usuario a la aplicación, donde puede registrarse o iniciar sesión utilizando sus credenciales. Una vez recibida la información, el sistema realiza las validaciones correspondientes y consulta la base de datos para verificar la existencia del usuario o almacenar un nuevo registro cuando corresponde. Si el proceso de autenticación resulta exitoso, el sistema genera la sesión del usuario y le permite acceder a las funcionalidades protegidas de la aplicación.

Posteriormente, el usuario consulta la información necesaria para realizar un agendamiento. Para ello, el sistema solicita a la base de datos los datos correspondientes a las campañas activas, los puntos de vacunación disponibles, las vacunas asociadas y los horarios habilitados para la atención. Una vez recuperada esta información, el sistema la presenta al usuario, quien puede seleccionar la campaña de interés, el punto de vacunación, la vacuna y el horario que mejor se ajusten a sus necesidades.

Tras confirmar la reserva, el sistema registra la solicitud mediante operaciones sobre la base de datos, almacenando la información correspondiente a la nueva cita y verificando la correcta persistencia de los datos. Finalmente, el sistema informa al usuario que el proceso ha concluido exitosamente, completando así el flujo principal de agendamiento representado en el modelo. Aunque durante la etapa de diseño se contempló la validación automática de ciertos datos personales mediante servicios externos, esta funcionalidad no fue incorporada en la implementación final debido al alcance del proyecto y a las limitaciones de tiempo, manteniéndose como una posibilidad de evolución futura.

Complementando este proceso, el esquema de seguridad define los mecanismos encargados de proteger la información y controlar el acceso a los distintos recursos del sistema. Desde el diseño se estableció que cada

usuario debe poseer una identidad única asociada a su RUT y autenticarse mediante credenciales antes de acceder a funcionalidades protegidas. Las contraseñas son almacenadas utilizando funciones de hash y, una vez verificadas las credenciales, el sistema genera un JSON Web Token (JWT) que representa la sesión autenticada del usuario.

Este token contiene información relevante como el identificador del usuario, el rol asignado y su tiempo de expiración, siendo enviado en cada solicitud dirigida a recursos protegidos. Antes de procesar cualquier operación, el sistema verifica la validez del token, su firma digital, la fecha de expiración y el estado del usuario, garantizando que únicamente las sesiones válidas puedan acceder a la aplicación.

Sobre este mecanismo de autenticación se implementó un modelo de control de acceso basado en roles, mediante el cual cada usuario dispone únicamente de los permisos asociados a sus responsabilidades dentro del sistema. Los ciudadanos pueden consultar información relacionada con su proceso de vacunación, los funcionarios cuentan con permisos para registrar vacunaciones y consultar antecedentes clínicos cuando corresponde, mientras que los coordinadores poseen privilegios administrativos para gestionar campañas, usuarios y centros de vacunación. Esta separación de responsabilidades permitió mantener un adecuado principio de mínimo privilegio y proteger las operaciones críticas del sistema.

Asimismo, el esquema contempla reglas específicas de autorización para garantizar que cada usuario únicamente pueda acceder a la información que le corresponde. Un ciudadano solo puede consultar su propio historial de vacunación, mientras que los funcionarios y coordinadores acceden únicamente a los recursos necesarios para el cumplimiento de sus funciones. Cualquier intento de realizar operaciones fuera de los permisos asignados provoca el rechazo inmediato de la solicitud.

Finalmente, el diseño incorpora mecanismos de auditoría que registran las operaciones relevantes realizadas dentro de la aplicación, incluyendo autenticaciones, consultas de historiales, creación y modificación de campañas, registro de vacunaciones y gestión de usuarios. Estos registros permiten mantener la trazabilidad de las acciones ejecutadas y facilitan el monitoreo del funcionamiento del sistema.

En conjunto, el modelo BPMN y el esquema de seguridad permitieron complementar el resto de los modelos desarrollados durante el proyecto. Mientras el primero representa la secuencia completa de actividades que siguen los distintos actores para llevar a cabo el proceso de vacunación, el segundo garantiza que dichas operaciones se ejecuten bajo mecanismos adecuados de autenticación, autorización y control de acceso. Ambos modelos sirvieron como guía durante la implementación, contribuyendo al desarrollo de una aplicación funcional, organizada y alineada con las buenas prácticas de ingeniería de software.

3. Resultados y aplicación

El sistema desarrollado constituye una plataforma completa para la gestión del proceso de vacunación, permitiendo administrar campañas, registrar usuarios, autenticar personal autorizado y gestionar citas de vacunación mediante una arquitectura cliente-servidor.

A diferencia de una aplicación monolítica, la solución implementada separa claramente la capa de presentación, la lógica de negocio y la persistencia de datos. Esta decisión permite disminuir el acoplamiento entre componentes, facilita la incorporación de nuevas funcionalidades y mejora la capacidad de mantenimiento del software a largo plazo.

El backend expone una API REST que centraliza toda la lógica del sistema, mientras que el frontend consume dicha API para presentar una interfaz moderna e interactiva al usuario. La persistencia de datos es administrada mediante Prisma ORM, el cual abstrae las operaciones SQL y permite trabajar con modelos tipados.

El resultado obtenido corresponde a una aplicación capaz de administrar el ciclo completo del proceso de vacunación, desde la autenticación del personal hasta la creación y gestión de campañas y la programación de citas, manteniendo una estructura organizada y preparada para futuras extensiones.

3.1. Flujo del sistema y modulos

La aplicación fue diseñada siguiendo una arquitectura modular, donde cada módulo representa una funcionalidad específica del dominio del problema. Esta organización evita concentrar toda la lógica en un único componente y permite que cada módulo sea responsable únicamente de sus propias operaciones.

El flujo general comienza cuando el usuario accede a la interfaz web. Desde este punto todas las solicitudes son enviadas al backend mediante peticiones HTTP hacia la API REST.

Cada petición sigue prácticamente el mismo recorrido:

1. La interfaz envía la solicitud.
2. El controlador correspondiente recibe la petición.
3. Se validan los datos recibidos.
4. El servicio ejecuta la lógica de negocio.
5. El repositorio interactúa con la base de datos mediante Prisma.
6. Los datos obtenidos son transformados mediante mappers y DTOs.
7. Finalmente se devuelve una respuesta estructurada al frontend.

Esta separación puede observarse claramente en la estructura del backend, donde cada módulo posee sus propios archivos de:

- Controller
- Service
- Repository
- Routes
- DTO
- Mapper
- Schema

Esta organización sigue un patrón por capas ampliamente utilizado en diversas aplicaciones, permitiendo que cada componente tenga una única responsabilidad.

3.1.1. Módulo de autenticación

El módulo de autenticación es el punto de entrada al sistema para los usuarios registrados.

Cuando un usuario intenta iniciar sesión, el controlador recibe las credenciales enviadas desde el frontend. Posteriormente el servicio consulta el repositorio para buscar el usuario correspondiente al correo electrónico ingresado.

Una vez recuperado el usuario desde la base de datos, la contraseña proporcionada es comparada con la contraseña cifrada almacenada utilizando la biblioteca bcrypt.

Si la autenticación es exitosa, el sistema genera un token JWT que contiene la información mínima necesaria del usuario (identificador, correo electrónico y rol). Dicho token será utilizado posteriormente para autenticar todas las solicitudes protegidas.

Esta estrategia evita almacenar sesiones directamente en el servidor, reduciendo la carga sobre éste y permitiendo una arquitectura completamente stateless.

3.1.2. Módulo de usuarios

El módulo de usuarios administra la información correspondiente a las personas autorizadas para utilizar el sistema.

La información de cada usuario se encuentra centralizada en la base de datos y es transformada mediante DTOs antes de ser enviada al cliente, evitando exponer información sensible como contraseñas cifradas u otros campos internos.

La existencia de este módulo permite desacoplar completamente la gestión de usuarios del proceso de autenticación.

3.1.3. Módulo de campañas

Este módulo administra todas las campañas de vacunación disponibles.

Las campañas contienen la información necesaria para organizar los distintos procesos de inmunización, incluyendo datos como nombre, descripción, fechas de inicio y término, población objetivo y demás información requerida por el sistema.

Toda la lógica relacionada con campañas se encuentra encapsulada en su propio servicio, permitiendo que futuras funcionalidades puedan incorporarse sin afectar el resto del sistema.

La existencia de este módulo permite además mantener una separación conceptual entre las campañas y las citas individuales, ya que múltiples citas pueden pertenecer a una misma campaña.

3.1.4. Módulo de citas

El módulo de citas constituye uno de los componentes principales de la aplicación.

Su responsabilidad consiste en administrar la programación de vacunaciones entre pacientes y campañas disponibles.

Cada cita posee su propio ciclo de vida, pudiendo ser creada, consultada, modificada o eliminada mediante los distintos endpoints implementados en la API.

La lógica de negocio asociada a las citas se encuentra completamente separada de la capa de persistencia, lo cual facilita futuras modificaciones como la incorporación de validaciones adicionales, restricciones horarias o algoritmos automáticos de asignación de horarios.

3.1.5. Organización interna del backend

Uno de los aspectos más relevantes de la implementación es la organización interna del backend.

Cada módulo posee una estructura prácticamente idéntica.

Los controladores reciben las solicitudes HTTP y únicamente coordinan el flujo de ejecución.

Los servicios contienen toda la lógica del negocio, siendo responsables de ejecutar las reglas del sistema.

Los repositorios encapsulan completamente el acceso a la base de datos utilizando Prisma ORM, evitando que el resto de la aplicación conozca detalles específicos sobre las consultas SQL.

Los DTO (Data Transfer Objects) definen exactamente qué información será enviada al cliente, mientras que los mappers realizan la transformación entre los modelos internos de Prisma y dichos objetos de transferencia.

Finalmente, los schemas validan la información recibida antes de que ésta alcance la lógica principal del sistema.

Esta división reduce considerablemente el acoplamiento entre componentes y facilita la realización de pruebas unitarias.

3.2. Patrones de diseño: ajustes respecto al diseño inicial

Durante la etapa de diseño se contempló la utilización de los patrones Factory, Observer y Proxy para resolver problemas específicos anticipados en el modelo conceptual. Sin embargo, durante la implementación se identificó que dichos problemas no se manifestaban de la forma prevista, o bien existían soluciones más simples y adecuadas al contexto real del proyecto, por lo que se optó por reemplazarlos por decisiones de diseño más acordes a la implementación final.

Factory: Este patrón fue pensado originalmente para instanciar distintos tipos de centros de vacunación con propiedades adicionales según su categoría. Durante la implementación, sin embargo, los puntos de vacunación se modelaron como una única entidad homogénea (`VaccinationPoint`), sin variaciones de comportamiento entre distintos tipos de centro. Al no existir jerarquías ni especializaciones que justificaran una lógica de creación condicional, mantener el patrón Factory habría introducido una capa de abstracción innecesaria sobre un proceso de creación que, en la práctica, es directo y uniforme para todos los centros.

Observer: Se había planteado para desacoplar la confirmación de una cita de sus efectos colaterales (envío de correo, registro en historial), permitiendo que un cambio de estado disparara múltiples notificaciones de forma independiente. En la implementación final, la confirmación de una cita se resolvió mediante una única operación en la capa de servicio (`confirmAppointment`) que actualiza el estado de la cita. Dado que el número de acciones asociadas a este evento es reducido y no se prevé, dentro del alcance del proyecto, la incorporación de múltiples suscriptores adicionales, se optó por una invocación directa desde el servicio en lugar de implementar todo el mecanismo de suscripción y notificación que exige el patrón Observer, evitando así una complejidad de diseño que no se traducía en un beneficio real para el sistema.

Proxy: Su propósito original era retrasar la instanciación de los servicios que dependen de la base de datos hasta el momento en que efectivamente se necesitaran. Al utilizar Prisma ORM como capa de persistencia, esta necesidad quedó cubierta por el propio cliente de Prisma, el cual ya administra de forma interna la conexión y el pool de conexiones a la base de datos de manera perezosa. Introducir un Virtual Proxy adicional sobre los servicios habría duplicado una responsabilidad que la herramienta de persistencia ya resuelve, agregando una capa de indirección sin aportar beneficios adicionales.

Como resultado de estos ajustes, la implementación incorporó otros patrones que respondieron de forma más directa a las necesidades reales del sistema:

Singleton: El cliente de Prisma (`config/prisma.ts`) se instancia una única vez y se comparte a través de todos los módulos del backend mediante su exportación como instancia única. Esto garantiza que exista una sola conexión administrada hacia la base de datos durante todo el ciclo de vida de la aplicación, evitando la creación de múltiples conexiones redundantes que podrían agotar los recursos disponibles del servidor de base de datos.

Repository: Cada módulo del backend encapsula el acceso a datos en una clase repositorio dedicada, que concentra exclusivamente las consultas realizadas mediante Prisma. Este patrón permite que los servicios trabajen contra una interfaz de acceso a datos sin conocer los detalles de las consultas subyacentes, facilitando tanto el mantenimiento del código como la eventual sustitución de la fuente de datos sin afectar la lógica de negocio.

Chain of Responsibility: El procesamiento de cada solicitud HTTP atraviesa una cadena de middlewares (`authenticate`, `authorize`, `validation`) antes de llegar al controlador correspondiente, seguida por un middleware de manejo centralizado de errores. Cada eslabón de la cadena decide de forma independiente si la solicitud puede continuar hacia el siguiente middleware o si debe ser interrumpida (por ejemplo, al rechazar una solicitud no autenticada), lo que permite añadir o modificar validaciones sin afectar el resto de la cadena de procesamiento.

Estos ajustes reflejan un proceso de diseño iterativo, donde las decisiones tomadas durante la etapa conceptual fueron contrastadas y refinadas a partir de las necesidades concretas surgidas durante la implementación, priorizando siempre la simplicidad y la correspondencia entre la complejidad del patrón utilizado y el problema efectivamente resuelto.

3.3. Seguridad y autenticación

La seguridad del sistema fue implementada utilizando múltiples mecanismos complementarios destinados a proteger tanto la información almacenada como el acceso a las funcionalidades del sistema.

Uno de los primeros mecanismos implementados corresponde al almacenamiento seguro de contraseñas.

En ningún momento las contraseñas son almacenadas en texto plano dentro de la base de datos. En su lugar, el sistema utiliza el algoritmo `bcrypt` para generar un hash irreversible antes de almacenar las credenciales del usuario.

Durante el proceso de autenticación, la contraseña ingresada nunca se compara directamente con el valor almacenado, sino que `bcrypt` realiza internamente la verificación utilizando el hash correspondiente.

Este procedimiento reduce significativamente el riesgo de exposición de credenciales ante una eventual filtración de la base de datos.

3.3.1. Autenticación mediante JWT

Luego de verificar correctamente las credenciales del usuario, el backend genera un JSON Web Token (JWT).

Este token contiene únicamente la información necesaria para identificar al usuario durante las solicitudes posteriores, incluyendo:

- identificador del usuario
- correo electrónico
- rol

El token es enviado al cliente, quien lo almacena temporalmente y lo incorpora posteriormente dentro del encabezado `Authorization` de cada petición protegida.

Gracias a este mecanismo el servidor no necesita mantener sesiones activas, permitiendo que la aplicación escale de manera más eficiente.

3.3.2. Protección del servidor

El servidor Express incorpora además diversos middlewares orientados a mejorar la seguridad y estabilidad de la aplicación.

Entre ellos destaca Helmet, encargado de establecer automáticamente múltiples cabeceras HTTP destinadas a reducir ataques comunes como clickjacking, sniffing de contenido o ciertas vulnerabilidades relacionadas con el navegador.

Asimismo, se utiliza CORS para controlar qué orígenes pueden comunicarse con la API, evitando solicitudes provenientes de dominios no autorizados.

El middleware Morgan registra las solicitudes realizadas al servidor, facilitando la detección de errores durante el desarrollo y proporcionando información útil para tareas de monitoreo.

3.3.3. Manejo centralizado de errores

Otro aspecto importante corresponde al manejo uniforme de errores.

La aplicación incorpora middlewares específicos para capturar rutas inexistentes y excepciones producidas durante la ejecución.

Esto permite evitar que errores internos lleguen directamente al cliente, entregando respuestas controladas y consistentes, además de simplificar considerablemente el mantenimiento del sistema.

3.4. Interfaz de usuario y Heurísticas

La interfaz fue desarrollada utilizando React junto con TypeScript, permitiendo construir una aplicación web moderna basada en componentes reutilizables.

La utilización de componentes favorece la reutilización de código y mantiene una separación clara entre la presentación y la lógica de interacción.

La estructura del proyecto organiza la interfaz en carpetas independientes para funcionalidades, componentes compartidos, servicios de acceso a la API y elementos de diseño, facilitando la mantenibilidad del código.

Desde el punto de vista de la experiencia de usuario, la aplicación busca mantener una navegación consistente y sencilla, permitiendo acceder rápidamente a las principales funcionalidades del sistema.

3.4.1. Aplicación de heurísticas de usabilidad

Durante el desarrollo pueden identificarse diversas heurísticas de usabilidad aplicadas en la interfaz.

Consistencia y estándares: La organización visual mantiene una estructura homogénea entre las distintas pantallas.

Los elementos de interacción presentan comportamientos similares, reduciendo la curva de aprendizaje del usuario.

Visibilidad del estado del sistema: Las operaciones realizadas mediante la interfaz generan respuestas visibles para el usuario, permitiendo conocer el resultado de las acciones ejecutadas.

Esto resulta especialmente importante durante procesos como autenticación, creación de campañas o gestión de citas.

Prevención de errores: Antes de enviar información al servidor, los datos son validados mediante esquemas definidos tanto en el frontend como en el backend.

La validación temprana evita que datos incorrectos lleguen a la lógica del negocio, disminuyendo considerablemente la probabilidad de errores durante la ejecución.

Correspondencia entre el sistema y el mundo real: Las funcionalidades fueron organizadas utilizando conceptos propios del dominio de vacunación.

Los usuarios interactúan con campañas, citas y registros de vacunación, terminología que coincide con los procesos reales utilizados por centros de salud.

Esto facilita la comprensión del sistema incluso para usuarios con conocimientos técnicos limitados.

Diseño minimalista: La interfaz evita presentar información innecesaria en cada pantalla.

Cada vista concentra únicamente los elementos requeridos para completar la tarea correspondiente, reduciendo la carga cognitiva del usuario y facilitando la navegación.

3.5. Arquitectura Cliente - Servidor

La solución implementa una arquitectura cliente-servidor claramente definida.

El frontend constituye el cliente encargado exclusivamente de la interacción con el usuario, mientras que el backend concentra toda la lógica del negocio y el acceso a los datos.

La comunicación entre ambos componentes se realiza mediante una API REST utilizando mensajes HTTP en formato JSON.

Esta arquitectura ofrece múltiples ventajas, entre ellas la posibilidad de sustituir cualquiera de las capas sin modificar el resto del sistema. Por ejemplo, sería posible desarrollar una aplicación móvil que consuma exactamente la misma API implementada para la versión web.

Además, la separación favorece el trabajo colaborativo entre equipos de desarrollo, permitiendo que frontend y backend evolucionen de forma paralela.

3.6. Persistencia de datos

La persistencia fue implementada utilizando Prisma ORM como capa de acceso a datos.

Prisma actúa como intermediario entre la aplicación y la base de datos, permitiendo manipular registros mediante objetos tipados en lugar de consultas SQL escritas manualmente.

Otra ventaja importante corresponde al uso de migraciones, las cuales permiten versionar los cambios realizados sobre el esquema de la base de datos.

Gracias a este mecanismo, cualquier modificación estructural queda registrada, facilitando la sincronización entre distintos entornos de desarrollo y reduciendo la posibilidad de inconsistencias durante el despliegue del sistema.

Asimismo, el proyecto incorpora un proceso de inicialización mediante *seed*, encargado de poblar automáticamente la base de datos con información inicial cuando resulta necesario, simplificando considerablemente las tareas de instalación y pruebas del sistema.

3.7. Integración de notificaciones mediante Brevo

Como parte de los requisitos de valor agregado orientados a mejorar la experiencia del usuario y la trazabilidad del proceso, se implementó un sistema de notificaciones automáticas por correo electrónico. Para lograr esto, se integró la plataforma **Brevo** como proveedor de servicios de mensajería transaccional.

La integración con Brevo se realizó a nivel del backend, encapsulando la lógica de comunicación dentro de un servicio dedicado. En el flujo del sistema, cuando un usuario agenda, modifica o cancela con éxito una cita de vacunación, el servicio principal, tras confirmar la persistencia de los cambios en la base de datos, invoca al servicio de notificaciones. Este último se comunica con la API REST de Brevo, autenticándose mediante una clave de acceso (*API Key*) almacenada de forma segura en las variables de entorno del servidor.

El uso de este proveedor externo fortalece la arquitectura del sistema y se alinea de forma consistente con el principio de separación de responsabilidades. Al externalizar el envío de correos, el backend mantiene su enfoque exclusivo en la lógica de negocio central y la gestión de campañas, mientras que el ciudadano recibe retroalimentación inmediata en su bandeja de entrada sobre las acciones realizadas en la plataforma, acercando el proyecto a los estándares de un entorno de producción real.

4. Conclusión

El desarrollo de este proyecto permitió transitar desde el análisis y modelado conceptual (requisitos, modelo de dominio, modelo entidad-relación, diagramas de comunicación, modelo C4 y esquema de seguridad) hacia la implementación de una aplicación funcional para la gestión de campañas de vacunación. El sistema resultante cubre el flujo principal definido en la etapa de diseño: administración de campañas, gestión de usuarios y roles, puntos de vacunación y horarios, agendamiento de citas y registro del historial de vacunación.

La arquitectura implementada, organizada en módulos con capas de controlador, servicio, repositorio, DTO y mapper, junto con la persistencia mediante Prisma ORM y PostgreSQL, mantuvo una correspondencia directa con lo definido en el diseño inicial, reflejando los patrones GRASP identificados en los diagramas de comunicación. La seguridad se resolvió mediante autenticación JWT, contraseñas cifradas con bcrypt y control de acceso por roles, mientras que el frontend en React y TypeScript incorporó heurísticas de usabilidad para ofrecer una experiencia consistente entre los distintos perfiles de usuario. Adicionalmente, se integró **Brevo** como proveedor externo de mensajería para el envío automático de correos electrónicos de confirmación, validando en la práctica que la arquitectura desacoplada del sistema permite incorporar servicios externos sin afectar el resto de los componentes.

En conjunto, el proyecto permitió aplicar de forma práctica los principios de diseño orientado a objetos y arquitectura de software abordados en el curso, obteniendo una solución modular y extensible, alineada con los requerimientos planteados en la etapa de diseño.